

從 Web Component 轉換為 Lit Element

來源：<https://codelabs.developers.google.com/codelabs/the-lit-path?hl=zh-tw>

步驟數：9

在本程式碼研究室中，您將瞭解 & 從頭開始建構 Web 元件，然後逐步強化為 Lit Element。

目錄

1. [簡介](#)
2. [設定及探索 Playground](#)
3. [定義自訂元素](#)
4. [Shadow DOM](#)
5. [HTML 範本](#)
6. [新增功能](#)
7. [Lit-html](#)
8. [LitElement](#)
9. [恭喜](#)

步驟 1 · 約 0 分鐘

1. 簡介

上次更新時間：2021 年 8 月 10 日

網頁元件

網頁元件是一組網頁平台 API，可讓您建立新的自訂、可重複使用且封裝的 HTML 標記，用於網頁和網頁應用程式。以 Web Component 標準建構的自訂元件和小工具可跨新式瀏覽器運作，且能與任何適用於 HTML 的 JavaScript 程式庫或框架搭配使用。

什麼是 Lit

[Lit](#) 是一個簡單的程式庫，可建構快速輕巧的網頁元件，適用於任何框架，或完全不使用框架。您可以使用 Lit 建構可共用的元件、應用程式、設計系統等。

Lit 提供 API，可簡化常見的 Web Components 工作，例如管理屬性、屬性和算繪。

課程內容

- 什麼是網頁元件
- 網頁元件的概念
- 如何建構網頁元件
- 什麼是 lit-html 和 LitElement
- Lit 在網頁元件上執行的作業

如果您只是想瞭解 Lit，而不是 Lit 的運作方式或幕後運作原理，建議您前往 lit.dev 瀏覽 Lit 說明文件網站

建構項目

- 原生的喜歡 / 不喜歡網頁元件
- 以 Lit 為基礎的「喜歡」/「不喜歡」Web Component

軟硬體需求

- 任何更新版的新型瀏覽器 (Chrome、Safari、Firefox、Chromium Edge)。網頁元件適用於所有新式瀏覽器，且 Microsoft Internet Explorer 11 和非 Chromium 版 Microsoft Edge 均提供 Polyfill。
- 瞭解 HTML、CSS、JavaScript 和 [Chrome 開發人員工具](#)。

您是否已瞭解 Web Components 的基本概念？請跳至步驟 7「lit-html」，瞭解 lit-html 和 LitElement 在 Web Components 基礎上執行的作業！

步驟 2 · 約 3 分鐘

2. 設定及探索 Playground

存取程式碼

本程式碼研究室會提供 Lit Playground 的連結，如下所示：

這個遊樂區是程式碼沙箱，完全在瀏覽器中執行。它可以編譯及執行 TypeScript 和 JavaScript 檔案，也能自動將匯入內容解析為節點模組。例如：

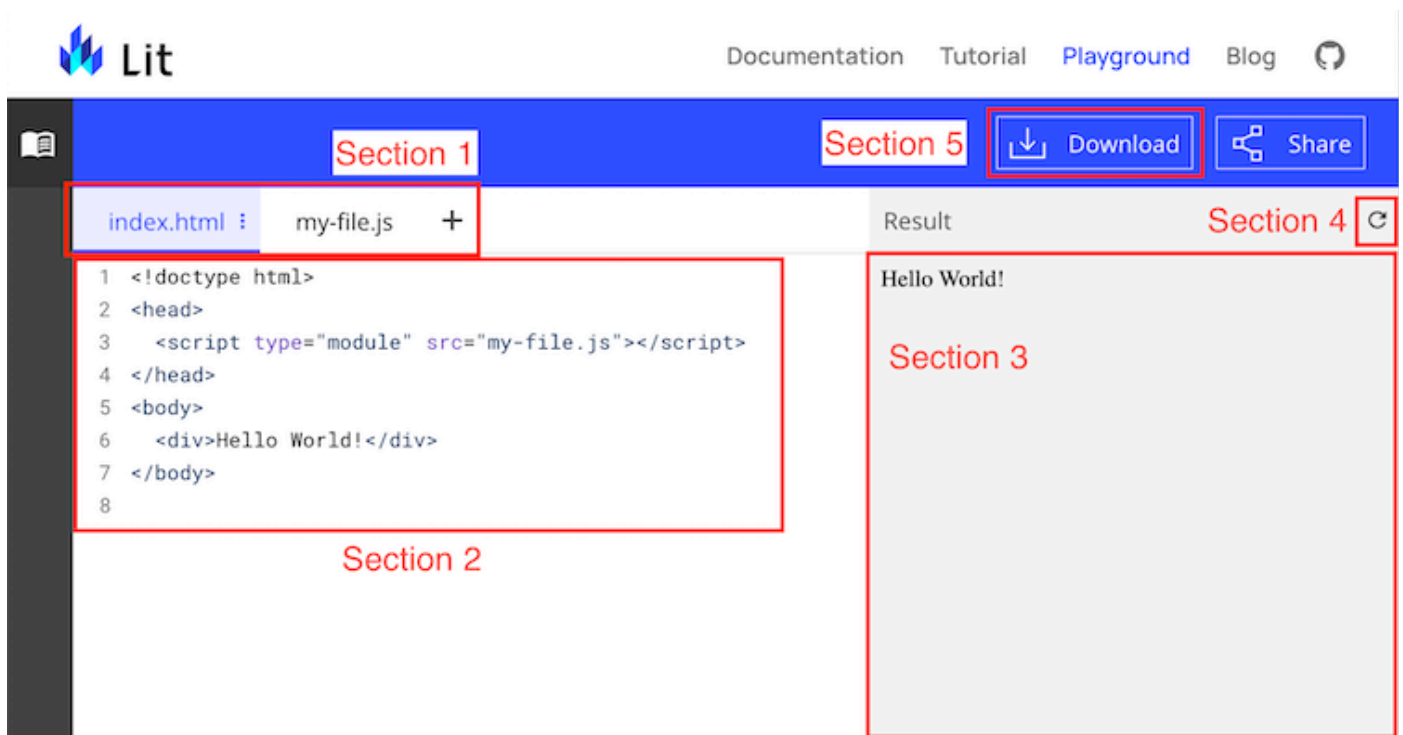
〇〇〇

```
// before
import './my-file.js';
import 'lit';

// after
import './my-file.js';
import 'https://unpkg.com/lit?module';
```

您可以在 Lit Playground 中完成整個教學課程，並將這些檢查點做為起點。如果您使用 VS Code，可以透過這些檢查點下載任何步驟的起始程式碼，並用來檢查您的成果。

探索點亮的遊樂場 UI



Lit 操場 UI 螢幕截圖會標示您在本程式碼研究室中使用的區段。

1. 檔案選取器。請注意加號按鈕...
2. 檔案編輯器。
3. 程式碼預覽。
4. 重新載入按鈕。

5. 「下載」按鈕。

您也可以使用 `ctrl + s` (Mac 則為 `cmd + s`) 將進度儲存為網址。這樣一來，你就能重新載入頁面，不會遺失進度，也能與他人分享代碼。

VS Code 設定 (進階)

如果您打算完全在瀏覽器中執行本程式碼研究室，可以略過這個步驟。

使用這項 VS Code 設定的好處如下：

- 檢查範本類型
- 範本智慧感應和自動完成

如果您已安裝 NPM、VS Code (含 [lit-plugin](#) 外掛程式)，且知道如何使用該環境，只要下載並啟動這些專案即可，方法如下：

- 按下下載按鈕
- 將 tar 檔案內容解壓縮至目錄
- 安裝可解析裸露模組指定符的開發伺服器 (Lit 團隊建議使用 [@web/dev-server](#))
 - 範例如下：`package.json`
- 執行開發伺服器並開啟瀏覽器 (如果您使用 `@web/dev-server`，可以改用 `npx web-dev-server --node-resolve --watch --open`)
 - 如果您使用範例 `package.json`，請使用 `npm run serve`

步驟 3 · 約 6 分鐘

3. 定義自訂元素

自訂元素

Web Components 是一組 4 個原生網頁 API。這 3 個子類型如下：

- ES 模組
- 自訂元素
- Shadow DOM
- HTML 範本

您已使用 ES 模組規格，可建立含有匯入和匯出項目的 JavaScript 模組，並透過 `<script type="module">` 載入網頁。

定義自訂元素

使用者可以透過自訂元素規格，使用 JavaScript 定義自己的 HTML 元素。名稱必須包含連字號 (-)，才能與原生瀏覽器元素區別。清除 `index.js` 檔案，並定義自訂元素類別：

index.js

```
class RatingElement extends HTMLElement {}

customElements.define('rating-element', RatingElement);
```

定義自訂元素時，請將擴充 `HTMLElement` 的類別與連字號標記名稱建立關聯。對 `customElements.define` 的呼叫會告知瀏覽器將類別 `RatingElement` 與 `tagName` `'rating-element'` 建立關聯。也就是說，文件中名為 `<rating-element>` 的每個元素都會與這個類別建立關聯。

由於 `customElements.define` 具有全域範圍，即使提供相同的類別定義，您目前也無法針對相同標記名稱呼叫多次。

在文件內文中放置 `<rating-element>`，看看會呈現什麼內容。

index.html

```
<body>
  <rating-element></rating-element>
</body>
```

現在查看輸出內容，您會發現沒有任何項目已算繪。這是正常情況，因為您尚未告知瀏覽器如何算繪 `<rating-element>`。如要確認自訂元素定義是否成功，請在 Chrome 開發人員版工具的元素選取器中選取 `<rating-element>`，然後在控制台中呼叫：

```
$0.constructor
```

這應該會輸出：

```
class RatingElement extends HTMLElement {}
```

自訂元素生命週期

自訂元素會附帶一組生命週期掛鉤。這 3 個子類型如下：

- `constructor`
- `connectedCallback`
- `disconnectedCallback`
- `attributeChangedCallback`
- `adoptedCallback`

首次建立元素時，系統會呼叫 `constructor`，例如呼叫 `document.createElement('rating-element')` 或 `new RatingElement()`。建構函式是設定元素的理想位置，但一般來說，基於元素「啟動」效能考量，不建議在建構函式中進行 DOM 操作。

當自訂元素附加至 DOM 時，系統會呼叫 `connectedCallback`。通常會在這裡進行初始 DOM 操作。

自訂元素從 DOM 移除後，系統會呼叫 `disconnectedCallback`。

使用者指定的任何屬性變更時，系統都會呼叫 `attributeChangedCallback(attrName, oldValue, newValue)`。

當自訂元素透過 `adoptNode` (例如 `HTMLTemplateElement`) 從另一個 `documentFragment` 採用至主要文件中時，系統會呼叫 `adoptedCallback`。

顯示 DOM

現在返回自訂元素，並將一些 DOM 與其建立關聯。元素附加至 DOM 時，請設定元素內容：

index.js

```

class RatingElement extends HTMLElement {
  constructor() {
    super();
    this.rating = 0;
  }
  connectedCallback() {
    this.innerHTML = `
      <style>
        rating-element {
          display: inline-flex;
          align-items: center;
        }
        rating-element button {
          background: transparent;
          border: none;
          cursor: pointer;
        }
      </style>
      <button class="thumb_down" >
        <svg xmlns="http://www.w3.org/2000/svg" height="24" viewBox="0 0 24 24" width="24">
<path d="M15 3H6c-.83 0-1.54.5-1.84 1.22l-3.02 7.05c-.09.23-.14.47-.14.73v2c0 1.19 2 2
2h6.31l-.95 4.57-.03.32c0 .41.17.79.44 1.06L9.83 23l6.59-6.59c.36-.36.58-.86.58-1.41V5c0-
1.1-.9-2-2-2zm4 0v12h4V3h-4z"/></svg>
      </button>
      <span class="rating">${this.rating}</span>
      <button class="thumb_up">
        <svg xmlns="http://www.w3.org/2000/svg" height="24" viewBox="0 0 24 24" width="24">
<path d="M1 21h4V9H1v12zm22-11c0-1.1-.9-2-2-2h-6.31l.95-4.57.03-.32c0-.41-.17-.79-.44-
1.06L14.17 1 7.59 7.59C7.22 7.95 7 8.45 7 9v10c0 1.19 2 2 2h9c.83 0 1.54-.5 1.84-1.22l3.02-
7.05c.09-.23.14-.47.14-.73v-2z"/></svg>
      </button>
    `;
  }
}

customElements.define('rating-element', RatingElement);

```

在 `constructor` 中，您會在元素上儲存名為 `rating` 的執行個體屬性。在 `connectedCallback` 中，您會將 DOM 子項新增至 `<rating-element>`，以顯示目前的評分，以及「喜歡」和「不喜歡」按鈕。

請注意，為了方便說明，您並未遵循這些控制項和輸入內容的無障礙最佳做法。

4. Shadow DOM

為何要使用 Shadow DOM ？

在上一個步驟中，您會發現插入的樣式標記中的選取器，會選取網頁上的任何評分元素和按鈕。這可能會導致樣式從元素溢出，並選取您可能不想設定樣式的其他節點。此外，這個自訂元素以外的其他樣式，可能會無意間為自訂元素內的節點設定樣式。舉例來說，請嘗試在主要文件的 head 中加入樣式標記：

index.html

```
<!DOCTYPE html>
<html>
  <head>
    <script src="./index.js" type="module"></script>
    <style>
      span {
        border: 1px solid red;
      }
    </style>
  </head>
  <body>
    <rating-element></rating-element>
  </body>
</html>
```

輸出內容中，分級的範圍應以紅色邊框方塊標示。這是微不足道的情況，但缺乏 DOM 封裝可能會導致更複雜的應用程式出現更大的問題。這時，Shadow DOM 就派上用場了。

附加 Shadow Root

將 Shadow Root 附加至元素，並在該根目錄內算繪 DOM：

index.js

```

class RatingElement extends HTMLElement {
  constructor() {
    super();
    this.rating = 0;
  }
  connectedCallback() {
    const shadowRoot = this.attachShadow({mode: 'open'});

    shadowRoot.innerHTML = `
      <style>
        :host {
          display: inline-flex;
          align-items: center;
        }
        button {
          background: transparent;
          border: none;
          cursor: pointer;
        }
      </style>
      <button class="thumb_down" >
        <svg xmlns="http://www.w3.org/2000/svg" height="24" viewBox="0 0 24 24" width="24">
<path d="M15 3H6c-.83 0-1.54.5-1.84 1.22l-3.02 7.05c-.09.23-.14.47-.14.73v2c0 1.19 2 2
2h6.31l-.95 4.57-.03.32c0 .41.17.79.44 1.06L9.83 23l6.59-6.59c.36-.36.58-.86.58-1.41V5c0-
1.1-.9-2-2-2zm4 0v12h4V3h-4z"/></svg>
      </button>
      <span class="rating">${this.rating}</span>
      <button class="thumb_up">
        <svg xmlns="http://www.w3.org/2000/svg" height="24" viewBox="0 0 24 24" width="24">
<path d="M1 21h4V9H1v12zm22-11c0-1.1-.9-2-2-2h-6.31l.95-4.57.03-.32c0-.41-.17-.79-.44-
1.06L14.17 1 7.59 7.59C7.22 7.95 7 8.45 7 9v10c0 1.19 2 2 2h9c.83 0 1.54-.5 1.84-1.22l3.02-
7.05c.09-.23.14-.47.14-.73v-2z"/></svg>
      </button>
    `;
  }
}

customElements.define('rating-element', RatingElement);

```

重新整理頁面後，您會發現主要文件中的樣式無法再選取 Shadow Root 內的節點。

你是怎麼辦到的？您在 `connectedCallback` 中呼叫了 `this.attachShadow`，將陰影根附加至元素。`open` 模式表示陰影內容可供檢查，且陰影根目錄也可透過 `this.shadowRoot` 存取。此外，請在 Chrome 檢查器中查看 Web 元件：

 Chrome 檢查器中的 DOM 樹狀結構。有一個 `<rating-element>`，其中包含 `#shadow-root (open)` 做為子項，以及該 `shadowroot` 內先前的 DOM。

您現在應該會看到可展開的陰影根，其中包含內容。`shadow root` 內的所有內容都稱為 Shadow DOM。如果您在 Chrome 開發人員工具中選取評分元素並呼叫 `$0.children`，會發現系統未傳回任何子項。這是因為 Shadow DOM 不會視為與直接子項相同的 DOM 樹狀結構，而是 *Shadow Tree*。

Shadow Tree 會將 CSS 選取器和 `querySelector` 範圍限定在本地陰影根層級，藉此提供 CSS 和 DOM 封裝。

Light DOM

實驗：將節點新增為 `<rating-element>` 的直接子項：

index.html

```
<rating-element>
  <div>
    This is the light DOM!
  </div>
</rating-element>
```

重新整理頁面，您會發現這個自訂元素 *Light DOM* 中的新 DOM 節點不會顯示在頁面上。這是因為 Shadow DOM 具有相關功能，可透過 `<slot>` 元素控制 Light DOM 節點投影至 Shadow DOM 的方式。

本程式碼研究室不會介紹 Light DOM 投影，但您可以在這篇 [MDN 文章](#) 或 [Lit 文件網站](#) 中進一步瞭解。

5. HTML 範本

使用範本的好處

使用 `innerHTML` 和範本字串常值 (未經過清除) 可能會導致指令碼插入的安全問題。過去的方法包括使用 `DocumentFragment`，但這些方法也會帶來其他問題，例如在定義範本時載入圖片和執行指令碼，以及造成重複使用障礙。這時 `<template>` 元素就派上用場了；範本提供惰性 DOM、高效能的節點複製方法，以及可重複使用的範本。

使用範本

接著，將元件轉換為使用 HTML 範本：

index.html

```
<body>
  <template id="rating-element-template">
    <style>
      :host {
        display: inline-flex;
        align-items: center;
      }
      button {
        background: transparent;
        border: none;
        cursor: pointer;
      }
    </style>
    <button class="thumb_down" >
      <svg xmlns="http://www.w3.org/2000/svg" height="24" viewBox="0 0 24 24" width="24"><path
d="M15 3H6c-.83 0-1.54.5-1.84 1.22l-3.02 7.05c-.09.23-.14.47-.14.73v2c0 1.19 2 2 2h6.31l-.95
4.57-.03.32c0 .41.17.79.44 1.06L9.83 23l6.59-6.59c.36-.36.58-.86.58-1.41V5c0-1.1-.9-2-2-2zm4
0v12h4V3h-4z"/></svg>
    </button>
    <span class="rating"></span>
    <button class="thumb_up">
      <svg xmlns="http://www.w3.org/2000/svg" height="24" viewBox="0 0 24 24" width="24"><path
d="M1 21h4V9H1v12zm22-11c0-1.1-.9-2-2-2h-6.31l-.95-4.57.03-.32c0-.41-.17-.79-.44-1.06L14.17 1
7.59 7.59C7.22 7.95 7 8.45 7 9v10c0 1.19 2 2 2h9c.83 0 1.54-.5 1.84-1.22l3.02-
7.05c.09-.23.14-.47.14-.73v-2z"/></svg>
    </button>
  </template>

  <rating-element>
    <div>
      This is the light DOM!
    </div>
  </rating-element>
</body>
```

您已將 DOM 內容移至主要文件 DOM 中的範本標記。現在重構自訂元素定義：

index.js

```
class RatingElement extends HTMLElement {
  constructor() {
    super();
    this.rating = 0;
  }
  connectedCallback() {
    const shadowRoot = this.attachShadow({mode: 'open'});
    const templateContent = document.getElementById('rating-element-template').content;
    const clonedContent = templateContent.cloneNode(true);
    shadowRoot.appendChild(clonedContent);

    this.shadowRoot.querySelector('.rating').innerText = this.rating;
  }
}

customElements.define('rating-element', RatingElement);
```

如要使用這個範本元素，請查詢範本、取得內容，然後使用 `templateContent.cloneNode` 複製這些節點，其中 `true` 引數會執行深層複製。接著，使用資料初始化 DOM。

恭喜！您現在已擁有網頁元件！但目前還沒有任何作用，因此接下來請新增一些功能。

步驟 6 · 約 9 分鐘

6. 新增功能

屬性繫結

目前，如要在評分元素上設定評分，唯一方法是建構元素、在物件上設定 `rating` 屬性，然後將其放在網頁上。很抱歉，原生 HTML 元素通常不是這樣運作。原生 HTML 元素通常會隨著屬性和屬性變更而更新。

新增下列程式碼行，讓自訂元素在 `rating` 屬性變更時更新檢視區塊：

index.js

```
constructor() {
  super();
  this._rating = 0;
}

set rating(value) {
  this._rating = value;
  if (!this.shadowRoot) {
    return;
  }

  const ratingEl = this.shadowRoot.querySelector('.rating');
  if (ratingEl) {
    ratingEl.innerText = this._rating;
  }
}

get rating() {
  return this._rating;
}
```

您會為評分屬性新增 setter 和 getter，然後更新評分元素的文字 (如有)。也就是說，如果您在元素上設定評分屬性，檢視畫面就會更新；請在開發人員工具控制台中快速測試！

屬性繫結

現在，請在屬性變更時更新檢視區塊；這與您設定 `<input value="newValue">` 時，輸入內容更新檢視區塊類似。幸好，Web Component 生命週期包含 `attributeChangedCallback`。新增下列程式碼行，更新評分：

index.js

```

static get observedAttributes() {
  return ['rating'];
}

attributeChangedCallback(attributeName, oldValue, newValue) {
  if (attributeName === 'rating') {
    const newRating = Number(newValue);
    this.rating = newRating;
  }
}

```

如要觸發 `attributeChangedCallback`，您必須為 `RatingElement.observedAttributes` which defines the attributes to be observed for changes 設定靜態 getter。接著在 DOM 中以宣告方式設定評分。立即試試：

index.html

```
<rating-element rating="5"></rating-element>
```

現在應該會以宣告方式更新評分！

按鈕功能

現在只差按鈕功能了。這個元件的行為應允許使用者提供單一的讚或倒讚評分，並向使用者提供視覺回饋。您可以使用一些事件監聽器和反映屬性實作這項功能，但請先附加下列程式碼行，更新樣式以提供視覺回饋：

index.html

```

<style>
...

:host([vote=up]) .thumb_up {
  fill: green;
}
:host([vote=down]) .thumb_down {
  fill: red;
}
</style>

```

在 Shadow DOM 中，`:host` 選取器是指 Shadow Root 所附加的節點或自訂元素。在本例中，如果 `vote` 屬性為 "up"，系統會將「喜歡」按鈕設為綠色，但如果 `vote` 為 "down"，then it will turn the thumb-down button red，現在，請為 `vote` 建立反映屬性 / 屬性，實作這項邏輯，做法與實作 `rating` 類似。先從屬性設定器和擷取器開始：

index.js

```
constructor() {
  super();
  this._rating = 0;
  this._vote = null;
}

set vote(newValue) {
  const oldValue = this._vote;
  if (newValue === oldValue) {
    return;
  }
  if (newValue === 'up') {
    if (oldValue === 'down') {
      this.rating += 2;
    } else {
      this.rating += 1;
    }
  } else if (newValue === 'down') {
    if (oldValue === 'up') {
      this.rating -= 2;
    } else {
      this.rating -= 1;
    }
  }
  this._vote = newValue;
  this.setAttribute('vote', newValue);
}

get vote() {
  return this._vote;
}
```

您會在 `constructor` 中使用 `null` 初始化 `_vote` 執行個體屬性，並在設定器中檢查新值是否不同。如果是，請相應調整評分，並務必使用 `this.setAttribute` 將 `vote` 屬性反映回主機。

請注意，這並非最有效率的寫法，但可做為網頁元件狀態運作方式的範例。

接著，設定屬性繫結：

index.js


```

static get observedAttributes() {
  return ['rating', 'vote'];
}

attributeChangedCallback(attributeName, oldValue, newValue) {
  if (attributeName === 'rating') {
    const newRating = Number(newValue);

    this.rating = newRating;
  } else if (attributeName === 'vote') {
    this.vote = newValue;
  }
}

```

同樣地，這與您使用 `rating` 屬性繫結時的程序相同：將 `vote` 新增至 `observedAttributes`，並在 `attributeChangedCallback` 中設定 `vote` 屬性。最後，新增一些點擊事件監聽器，為按鈕提供功能！

index.js

```

constructor() {
  super();
  this._rating = 0;
  this._vote = null;
  this._boundOnUpClick = this._onUpClick.bind(this);
  this._boundOnDownClick = this._onDownClick.bind(this);
}

connectedCallback() {
  ...
  this.shadowRoot.querySelector('.thumb_up')
    .addEventListener('click', this._boundOnUpClick);
  this.shadowRoot.querySelector('.thumb_down')
    .addEventListener('click', this._boundOnDownClick);
}

disconnectedCallback() {
  this.shadowRoot.querySelector('.thumb_up')
    .removeEventListener('click', this._boundOnUpClick);
  this.shadowRoot.querySelector('.thumb_down')
    .removeEventListener('click', this._boundOnDownClick);
}

_onUpClick() {
  this.vote = 'up';
}

_onDownClick() {
  this.vote = 'down';
}

```

在 `constructor` 中，您會將一些點擊事件監聽器繫結至元素，並保留參照。在 `connectedCallback` 中，您會監聽按鈕的點擊事件。在 `disconnectedCallback` 中，您會清理這些監聽器，並在點擊監聽器本身上適當設定 `vote`。

恭喜！您現在已擁有功能齊全的網頁元件，不妨點選幾個按鈕試試！現在的問題是，我的 JS 檔案已達到 96 行，HTML 檔案則有 43 行，而且程式碼相當冗長，對於這麼簡單的元件來說，這類程式碼是必要的。這時，Google 的 Lit 專案就能派上用場！

7. Lit-html

程式碼檢查點

為什麼要使用 lit-html

首先，`<template>` 標記實用且效能良好，但並未與元件的邏輯封裝在一起，因此難以將範本與其餘邏輯一併發布。此外，範本元素的使用方式本質上會產生命令式程式碼，與宣告式編碼模式相比，這在許多情況下會導致程式碼可讀性較差。

這時，lit-html 就派上用場了！Lit HTML 是 Lit 的轉譯系統，可讓您在 JavaScript 中編寫 HTML 範本，然後有效率地算繪及重新算繪這些範本和資料，以建立及更新 DOM。這與熱門的 JSX 和 VDOM 程式庫類似，但可在瀏覽器中以原生方式執行，且在許多情況下效率更高。

如要進一步瞭解我們不會介紹的 Lit 範本功能 (例如範本重複和屬性繫結)，請前往 [Lit 說明文件](#) 網站。

使用 Lit HTML

接著，請遷移原生 Web 元件 `rating-element`，改用 Lit 範本，也就是使用 [標記範本字面值](#)，這類函式會以特殊語法將範本字串做為引數。Lit 接著會使用幕後的範本元素，提供快速算繪功能，以及一些安全防護的清除功能。首先，請將

`<template>` 中的 `index.html` 遷移至 Lit 範本，方法是在 `webcomponent` 中新增 `render()` 方法：

並非所有程式設計環境 (例如 Google 程式碼研究室) 都提供標記範本常值語法醒目顯示功能。如果您使用 Visual Studio Code 編寫標記範本字面值，建議下載 [lit-plugin](#) 外掛程式，以便使用語法螢光標示和其他網頁元件功能，或直接使用 [lit.dev 試驗場](#)！

index.js

```

// Dont forget to import from Lit!
import {render, html} from 'lit';

class RatingElement extends HTMLElement {
  ...
  render() {
    if (!this.shadowRoot) {
      return;
    }

    const template = html`
      <style>
        :host {
          display: inline-flex;
          align-items: center;
        }
        button {
          background: transparent;
          border: none;
          cursor: pointer;
        }

        :host([vote=up]) .thumb_up {
          fill: green;
        }

        :host([vote=down]) .thumb_down {
          fill: red;
        }
      </style>
      <button class="thumb_down">
        <svg xmlns="http://www.w3.org/2000/svg" height="24" viewBox="0 0 24 24" width="24">
<path d="M15 3H6c-.83 0-1.54-.5-1.84 1.22l-3.02 7.05c-.09.23-.14.47-.14.73v2c0 1.1.9 2 2
2h6.31l-.95 4.57-.03.32c0 .41.17.79.44 1.06L9.83 23l6.59-6.59c.36-.36.58-.86.58-1.41V5c0-
1.1-.9-2-2-2zm4 0v12h4V3h-4z"/></svg>
      </button>
      <span class="rating">${this.rating}</span>
      <button class="thumb_up">
        <svg xmlns="http://www.w3.org/2000/svg" height="24" viewBox="0 0 24 24" width="24">
<path d="M1 21h4V9H1v12zm22-11c0-1.1-.9-2-2-2h-6.31l.95-4.57.03-.32c0-.41-.17-.79-.44-
1.06L14.17 1 7.59 7.59C7.22 7.95 7 8.45 7 9v10c0 1.1.9 2 2 2h9c.83 0 1.54-.5 1.84-1.22l3.02-
7.05c.09-.23.14-.47.14-.73v-2z"/></svg>
      </button>`;

    render(template, this.shadowRoot);
  }
}

```

你也可以從 `index.html` 刪除範本。在這個算繪方法中，您定義了名為 `template` 的變數，並叫用 `html` 標記的範本常值函式。您也會發現，您已使用 `${...}` 的範本常值插補語法，在 `span.rating` 元素內執行簡單的資料繫結。也就是說，您最終將不再需要強制更新該節點。此外，您會呼叫 `lit render` 方法，將範本同步算繪至陰影根目錄。

遷移至宣告式語法

現在您已移除 `<template>` 元素，請重構程式碼，改為呼叫新定義的 `render` 方法。您可以先運用 `lit` 的事件監聽器繫結，清除監聽器程式碼：

index.js

```
<button
  class="thumb_down"
  @click=${() => {this.vote = 'down'}}>
...
<button
  class="thumb_up"
  @click=${() => {this.vote = 'up'}}>
```

`lit` 範本可使用 `@EVENT_NAME` 繫結語法，將事件監聽器新增至節點。在本例中，每當點擊這些按鈕時，您都會更新 `vote` 屬性。

如要進一步瞭解 `lit` 繫結語法，請前往 [Lit 說明文件網站](#)。

接著，清理 `constructor`、`connectedCallback` 和 `disconnectedCallback` 中的事件監聽器初始化程式碼：

index.js

```
constructor() {
  super();
  this._rating = 0;
  this._vote = null;
}

connectedCallback() {
  this.attachShadow({mode: 'open'});
  this.render();
}

// remove disconnectedCallback and _onUpClick and _onDownClick
```

您已從所有三個回呼中移除點擊事件監聽器邏輯，甚至完全移除 `disconnectedCallback`！您也可以從 `connectedCallback` 移除所有 DOM 初始化程式碼，讓程式碼看起來更簡潔。這也表示您可以擺脫 `_onUpClick` 和 `_onDownClick` 監聽器方法！

最後，請更新屬性設定器，使用 `render` 方法，以便在屬性或屬性變更時更新 DOM：

index.js

```

set rating(value) {
  this._rating = value;
  this.render();
}

...

set vote(newValue) {
  const oldValue = this._vote;
  if (newValue === oldValue) {
    return;
  }

  if (newValue === 'up') {
    if (oldValue === 'down') {
      this.rating += 2;
    } else {
      this.rating += 1;
    }
  } else if (newValue === 'down') {
    if (oldValue === 'up') {
      this.rating -= 2;
    } else {
      this.rating -= 1;
    }
  }

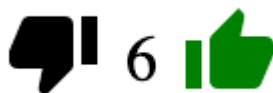
  this._vote = newValue;
  this.setAttribute('vote', newValue);
  // add render method
  this.render();
}

```

請注意，這並非最有效率的寫法，但其複雜度旨在說明 Web 元件和 Lit 生命週期的功能。

您可以在這裡從 `rating` 設定器移除 DOM 更新邏輯，並從 `vote` 設定器新增對 `render` 的呼叫。現在您可以查看繫結和事件監聽器的套用位置，因此範本更容易閱讀。

重新整理頁面，您應該會看到可正常運作的評分按鈕，按下「讚」按鈕時應如下所示！



8. LitElement

為何選擇 LitElement

程式碼仍存在一些問題。首先，如果您變更 `vote` 屬性或屬性，可能會變更 `rating` 屬性，導致 `render` 呼叫兩次。儘管重複呼叫算繪本質上是無運算且有效率的，但 JavaScript VM 仍會花時間同步呼叫該函式兩次。其次，新增屬性和屬性很麻煩，因為需要大量樣板程式碼。這時 `LitElement` 就能派上用場！

`LitElement` 是 Lit 的基礎類別，用於建立快速輕巧的網頁元件，可在各種架構和環境中使用。接著，請將實作項目變更為使用 `LitElement`，看看 `LitElement` 能為我們做些什麼！`rating-element`

使用 LitElement

首先，請從 `lit` 套件匯入並子類別化 `LitElement` 基礎類別：

index.js

```
import {LitElement, html, css} from 'lit';

class RatingElement extends LitElement {
  // remove connectedCallback()
  ...
}
```

您匯入 `LitElement`，這是 `rating-element` 的新基礎類別。接著，您會保留 `html` 匯入內容，最後 `css` 則可讓我們定義 CSS 標記的範本常值，以用於 CSS 數學、範本和其他幕後功能。

如需樣式功能的完整清單，請前往 Lit [說明文件網站](#)。

接著，將樣式從算繪方法移至 Lit 的靜態樣式表：

index.js

```
class RatingElement extends LitElement {
  static get styles() {
    return css`
      :host {
        display: inline-flex;
        align-items: center;
      }
      button {
        background: transparent;
        border: none;
        cursor: pointer;
      }

      :host([vote=up]) .thumb_up {
        fill: green;
      }

      :host([vote=down]) .thumb_down {
        fill: red;
      }
    `;
  }
  ...
}
```

大多數樣式都位於 Lit 中。Lit 會採用這些樣式，並使用 *Constructable Stylesheets* 等瀏覽器功能，加快算繪速度，並視需要透過舊版瀏覽器的 Web Components Polyfill 傳遞樣式。

生命週期

除了原生 Web 元件回呼之外，Lit 還提供一組算繪生命週期回呼方法。當宣告的 Lit 屬性變更時，就會觸發這些回呼。

如要進一步瞭解 Lit 算繪生命週期，請造訪 [Lit 說明文件網站](#)。

如要使用這項功能，您必須靜態宣告哪些屬性會觸發算繪生命週期。

index.js


```
static get properties() {
  return {
    rating: {
      type: Number,
    },
    vote: {
      type: String,
      reflect: true,
    }
  };
}

// remove observedAttributes() and attributeChangedCallback()
// remove set rating() get rating()
```

在此，您定義 `rating` 和 `vote` 會觸發 `LitElement` 算繪生命週期，並定義用於將字串屬性轉換為屬性的型別。

如要將複雜物件傳遞至 `LitElement`，可以將其定義為 `type: Object`，但通常不建議將物件當做字串屬性傳遞，而是當做屬性傳遞。例如：

```
document.querySelector('user-profile').user = this.user;
```

如果您使用 `lit-html`，請輸入：

```
<user-profile .user=${this.user}></user-profile>
```

更好的做法是將物件分散到網頁元件中，以宣告方式呈現。例如：

```
<user-profile .name=${this.user.name} .age=${this.user.age}>
  ${this.user.family.map(member => html`
    <family-member
      .name=${member.name}
      .relation=${member.relation}>
    </family-member>`)}
</user-profile>
```

此外，`vote` 屬性上的 `reflect` 旗標會自動更新您在 `vote` setter 中手動觸發的主機元素 `vote` 屬性。

現在您已擁有靜態屬性區塊，可以移除所有屬性和屬性算繪更新邏輯。也就是說，您可以移除下列方法：

- `connectedCallback`
- `observedAttributes`
- `attributeChangedCallback`
- `rating` (setter 和 getter)
- `vote` (setter 和 getter，但保留 setter 的變更邏輯)

您保留的是 `constructor`，並新增 `willUpdate` 生命週期方法：

index.js

```

constructor() {
  super();
  this.rating = 0;
  this.vote = null;
}

willUpdate(changedProps) {
  if (changedProps.has('vote')) {
    const newValue = this.vote;
    const oldValue = changedProps.get('vote');

    if (newValue === 'up') {
      if (oldValue === 'down') {
        this.rating += 2;
      } else {
        this.rating += 1;
      }
    } else if (newValue === 'down') {
      if (oldValue === 'up') {
        this.rating -= 2;
      } else {
        this.rating -= 1;
      }
    }
  }
}

// remove set vote() and get vote()

```

在這裡，您只需初始化 `rating` 和 `vote`，然後將 `vote` 設定器邏輯移至 `willUpdate` 生命週期方法。每當任何更新屬性變更時，系統都會在 `render` 之前呼叫 `willUpdate` 方法，因為 `LitElement` 會批次處理屬性變更，並非同步進行算繪。`willUpdate` 中反應式屬性 (例如 `this.rating`) 的變更不會觸發不必要的 `render` 生命週期呼叫。

最後，`render` 是 `LitElement` 生命週期方法，需要我們傳回 Lit 範本：

index.js

```

render() {
  return html`
    <button
      class="thumb_down"
      @click=${() => {this.vote = 'down'}}>
      <svg xmlns="http://www.w3.org/2000/svg" height="24" viewBox="0 0 24 24" width="24">
<path d="M15 3H6c-.83 0-1.54.5-1.84 1.22l-3.02 7.05c-.09.23-.14.47-.14.73v2c0 1.19 2 2
2h6.31l-.95 4.57-.03.32c0 .41.17.79.44 1.06L9.83 23l6.59-6.59c.36-.36.58-.86.58-1.41V5c0-
1.1-.9-2-2-2zm4 0v12h4V3h-4z"/></svg>
    </button>
    <span class="rating">${this.rating}</span>
    <button
      class="thumb_up"
      @click=${() => {this.vote = 'up'}}>
      <svg xmlns="http://www.w3.org/2000/svg" height="24" viewBox="0 0 24 24" width="24">
<path d="M1 21h4V9H1v12zm22-11c0-1.1-.9-2-2-2h-6.31l.95-4.57.03-.32c0-.41-.17-.79-.44-
1.06L14.17 1 7.59 7.59C7.22 7.95 7 8.45 7 9v10c0 1.19 2 2 2h9c.83 0 1.54-.5 1.84-1.22l3.02-
7.05c.09-.23.14-.47.14-.73v-2z"/></svg>
    </button>`;
}

```

您不必再檢查陰影根，也不必再呼叫先前從 'lit' 套件匯入的 `render` 函式。

現在預覽畫面中應該會顯示您的元素，請點選該元素！

9. 恭喜

恭喜！您已從頭開始建構 Web Component，並將其演變為 LitElement！

Lit 非常小巧 (縮小並壓縮後小於 5 KB)，速度極快，而且寫程式非常有趣！您可以製作供其他架構使用的元件，也可以使用此架構建構功能齊全的應用程式！

您現在已瞭解什麼是網頁元件、如何建構網頁元件，以及 Lit 如何簡化建構程序！

程式碼檢查點

要對照我們的最終程式碼檢查您的程式碼嗎？如要比較，請按[這裡](#)。

後續步驟

歡迎參閱其他程式碼研究室！

- [適用於 React 開發人員的 Lit](#)
- [使用 lit-element 建構 Brick Viewer](#)
- [使用 lit-element 建構故事元件](#)

其他資訊

- [Lit 互動式教學課程](#)
- [The Lit Docs](#)
- [Open Web Components](#) - 社群經營的指引和工具社群
- [WebComponents.dev](#) - 在所有已知架構中建立網頁元件

社群

- [Lit and Friends Slack](#) - 最大的網頁元件社群
 - [Twitter 上的@buildWithLit](#) - 建立 Lit 的團隊的 Twitter 帳戶
 - [Web Components SF](#) - 舊金山網頁元件聚會
-